NexFlow

ORCHESTRATING SUPREME MARKET CLARITY

02 — Full Tech Stack

the actual stack of every artifact in the NexFlow monorepo, with version numbers from real `package.json` files. Use this as the source of truth for build, deploy, and dev decisions. Section 8 reconciles where the original feasibility study disagrees with the codebase.

Contents

> **Document scope:** the actual stack of every artifact in the NexFlow monorepo, with version numbers from real `package.json` files. Use this as the source of truth for build, deploy, and dev decisions. Section 8 reconciles where the original feasibility study disagrees with the codebase.

---

1. Repository topology

NexFlow is a **pnpm workspace monorepo**. Three high-level zones:

```
artifacts/                ← deployable applications (each one workflow-bound)
├── nexflow/              ← Web app (React + Vite)
├── mobile/               ← Mobile app (Expo / React Native)
├── api-server/           ← Express 5 API
├── marketing-demo-video/ ← 90 s animated demo (React video artifact)
└── mockup-sandbox/       ← Internal design canvas

lib/                      ← Shared libraries (composite TS, emit declarations)
├── db/                   ← Drizzle schema + migrations
├── api-spec/             ← OpenAPI spec + Orval codegen
├── api-zod/              ← Generated Zod schemas
└── api-client-react/     ← Generated React Query hooks

scripts/                  ← Workspace utility scripts
pnpm-workspace.yaml       ← Catalog (pinned versions) + package discovery
tsconfig.base.json        ← Shared strict TS defaults
tsconfig.json             ← Solution file for composite libs only
```

The catalog in `pnpm-workspace.yaml` pins the cross-cutting packages (`react`, `framer-motion`, `tailwindcss`, `drizzle-orm`, `vite`, `@tanstack/react-query`, `@tailwindcss/vite`, `@vitejs/plugin-react`, `lucide-react`, `clsx`, `tailwind-merge`, `class-variance-authority`, `zod`, `@types/node`, `@types/react`, `@types/react-dom`, the three `@replit/vite-plugin-*` ones).

---

2. Web — `artifacts/nexflow`

Frontend dashboard. React + Vite single-page app, served as a static bundle in production behind the shared workspace proxy.

LAYER	CHOICE	VERSION
Runtime	Node.js (build-time only)	24.x
Framework	React	catalog (currently 19.x)
Bundler / dev server	Vite	catalog
Styles	Tailwind CSS + `@tailwindcss/vite`	catalog
Component primitives	shadcn/ui (Radix-based)	`@radix-ui/react-*` (≈30 packages)
Animation	Framer Motion	catalog
Icons	Lucide React + `react-icons`	catalog / 5.4
Routing	wouter	3.3.5
Forms	react-hook-form + `@hookform/resolvers` + Zod	7.55 / 3.10 / catalog
Data fetching	`@tanstack/react-query` (catalog) via generated hooks from `lib/api-client-react`	catalog
Charts	Recharts	2.15.2
Utility	`clsx`, `tailwind-merge`, `class-variance-authority`, `cmdk`, `date-fns`, `embla-carousel-react`, `vaul`, `react-day-picker`, `next-themes`, `sonner`, `tw-animate-css`	as in `package.json`
Schema	Zod (catalog)	catalog
Replit dev plugins	`@replit/vite-plugin-cartographer`, `vite-plugin-dev-banner`, `vite-plugin-runtime-error-modal`	catalog

Build command: `pnpm --filter @workspace/nexflow run build` (alias for `vite build`). Dev command: `pnpm --filter @workspace/nexflow run dev` — wired into the workflow `artifacts/nexflow: web`.

---

3. Mobile — `artifacts/mobile`

Expo / React Native app. SDK 54, React Native 0.81. Uses Expo Router for file-based navigation.

LAYER	CHOICE	VERSION
Runtime	Node.js (build-time)	24.x
Framework	React Native	0.81.5
Tooling	Expo SDK	~54.0.27
Routing	expo-router	~6.0.17
Fonts	`@expo-google-fonts/inter`	^0.4.0
Async storage	`@react-native-async-storage/async-storage`	2.2.0
Data fetching	`@tanstack/react-query` (catalog) via shared `@workspace/api-client-react`	catalog
Visual / motion	`expo-blur`, `expo-glass-effect`, `expo-linear-gradient`, `expo-haptics`, `react-native-reanimated`, `react-native-gesture-handler`, `react-native-keyboard-controller`	per `package.json`
Native APIs	`expo-camera` indirectly, `expo-image-picker` 17, `expo-location` 19, `expo-web-browser` 15	per `package.json`
SVG / icons	`react-native-svg` 15.12, `@expo/vector-icons` 15	per `package.json`
React-compiler	`babel-plugin-react-compiler` (beta)	beta
Web target	`react-native-web`	^0.21

Dev wiring (`pnpm --filter @workspace/mobile run dev`) sets the Expo tunnel via `$REPLIT_EXPO_DEV_DOMAIN` , `$REPLIT_DEV_DOMAIN` , and `$REPL_ID` . The mobile workflow (`artifacts/mobile: expo`) keeps it running.

---

4. API server — `artifacts/api-server`

Express 5 REST API. ESM-first, esbuild-bundled into a single `dist/index.mjs` artifact at startup.

LAYER	CHOICE	VERSION
Runtime	Node.js	24.x
Framework	Express	^5
HTTP CORS	cors	^2
Cookies	cookie-parser	^1.4.7
Logging	pino + pino-http (+ `pino-pretty` in dev)	^9 / ^10 / ^13
Schema	Zod via `@workspace/api-zod` (catalog)	catalog
ORM	Drizzle (`drizzle-orm`) via `@workspace/db`	catalog
AI client	`openai` (used against both OpenRouter and OpenAI baseURLs)	^4.95
Email	`resend`	^4.0.1
Build	esbuild + `esbuild-plugin-pino`	^0.27 / ^2.3
Type check	`tsc -p tsconfig.json --noEmit`	shared

Source layout (`artifacts/api-server/src/`): - `index.ts` — entry; reads `PORT` and starts the listener after `autoSeed()` . - `app.ts` — Express setup: `pino-http` , permissive CORS, JSON+text body parsing up to 25 MB, mounts `router` under `/api` . - `routes/index.ts` — central router; mounts 30+ subrouters (see §5). - `lib/ai.ts` — `aiChat` and `aiJson` with OpenRouter-preferred / OpenAI-fallback. - `lib/email.ts` — Resend client; pulls credentials from the Replit Connectors API. - `lib/autoSeed.ts` — seeds the DB on cold start with sample CRM data. - `lib/logger.ts` — pino singleton.

---

5. Routes mounted on `/api`

(Source: `artifacts/api-server/src/routes/index.ts` .)

```
GET  /api/healthz          (health)
/api/contacts              CRUD + AI enrichment + owner join
/api/companies             CRUD + intelligence cache
/api/deals                 CRUD + stage-change automation hooks
/api/activities            unified timeline (call, email, whatsapp, task, …)
/api/signals               buying-intent feed
/api/segments              dynamic + saved
/api/calls                 call history + transcripts + AI insights
/api/scripts               sales scripts (en/ar/both)
/api/notifications         in-app bell feed
/api/dashboard             default dashboard payload
/api/dashboards            user-built dashboards + widgets
/api/analytics             aggregations for the analytics module
/api/ai                    chat, briefing, JSON helpers
/api/ai (extras)           extra AI endpoints (briefing, redaction, etc.)
/api/users                 reps + roles
/api/properties            custom-property registry
/api/lists                 static lists + members
/api/views                 saved views
/api/automations           rules
/api/campaigns             marketing campaigns + sequences
/api/agents                user-built AI agents + runs
/api/tracking              email pixel tracking
/api/admin                 seed-extra (dev-only re-seed)
/api/business-cards        OCR scan endpoints
/api/playbooks             sales-playbook CRUD
/api/activity-capture      auto-log emails / calls
/api/attribution           multi-touch attribution
/api/health-scores         account engagement scoring
/api/redaction             call-recording PII redaction
/api/dedup                 deduplication scanner
/api/marketing             marketing extras (audiences etc.)
/api/power-dialer          AI-prioritized outbound queue
/api/lead-enrich           single-shot enrichment
```

6. Shared libraries — `lib/`

LIB	PURPOSE	NOTES
`lib/db`	Drizzle schema + migrations + DB client. Single `schema/index.ts` (≈580 lines) with all tables.	Composite TS lib. Run migrations with `pnpm --filter @workspace/db run …`.
`lib/api-spec`	OpenAPI `openapi.yaml` + Orval codegen config.	`pnpm --filter @workspace/api-spec run codegen` regenerates Zod schemas + React Query hooks.
`lib/api-zod`	Generated Zod schemas (mirror of OpenAPI components).	Imported by both server (validation) and clients (forms).
`lib/api-client-react`	Generated React Query hooks.	Used by the web app and (selectively) by the mobile app.

Type-checking model: libs are `composite: true`, leaf workspace packages run `tsc --noEmit`. Root `pnpm run typecheck` builds libs first then checks every leaf.

---

7. AI orchestration — concrete

Implemented entirely in `artifacts/api-server/src/lib/ai.ts`.

- **Two providers, one client:** uses the `openai` SDK pointed at two different `baseURL`s.
- `process.env.AI_INTEGRATIONS_OPENROUTER_BASE_URL` + `AI_INTEGRATIONS_OPENROUTER_API_KEY` for **OpenRouter** (preferred — multi-provider).
- `process.env.AI_INTEGRATIONS_OPENAI_BASE_URL` + `AI_INTEGRATIONS_OPENAI_API_KEY` for **OpenAI** (fallback).
- **Provider-to-model map** (`providerModelMap` in `ai.ts`): | `provider` arg | model used (via OpenRouter) | | --- | --- | | `openai` | `openai/gpt-4o-mini` | | `anthropic` | `anthropic/claude-3.5-sonnet` | | `gemini` | `google/gemini-2.0-flash-001` | | `perplexity` | `perplexity/llama-3.1-sonar-large-128k-online` | | `auto` (default) | `openai/gpt-4o-mini` |

When OpenRouter is unavailable, the client falls back to OpenAI and forces `gpt-4o-mini`.

- **Two helpers exposed:**
- `aiChat({ system, user, provider, model, json, maxTokens })` — returns `string`. Uses `response_format: { type: "json_object" }` when `json: true`. Crashes are swallowed (returns `""`) so routes never throw because of AI.
- `aiJson<T>({ system, user, provider, model, fallback })` — wraps `aiChat({ json: true })`, parses the result, and returns `fallback` on parse failure / empty response.

- **Voice features** — speech-to-text and text-to-speech run **in the browser** today, via the Web Speech API (`SpeechRecognition` + `SpeechSynthesisUtterance`) inside `artifacts/nexflow/src/components/VoiceCallModal.tsx` . Each captured utterance is POSTed to `POST /api/calls/voice-response` , which calls the same chat-completions client (OpenRouter / OpenAI) for the AI reply text — the route does **not** call OpenAI Whisper or any server-side audio transcription endpoint. There is no `openai.audio.transcriptions.create` call anywhere in `artifacts/api-server/` . Server-side Whisper / dedicated TTS providers are a future swap (see §13 voice row).
- **Image generation** (used in the Marketing AI module) — `gpt-image-1` and DALL·E 3 through the same OpenAI client.

---

8. Communications integrations

CHANNEL	PROVIDER	STATUS IN CODE
Email (transactional + campaigns)	**Resend**	Live — `artifacts/api-server/src/lib/email.ts` . Pulls the API key from the Replit `resend` connector, falls back to `RESEND_API_KEY` . Sender defaults to `NexFlow <onboarding@resend.dev>` . Tracking pixels emitted by the `/api/tracking` route.
WhatsApp Business	Meta Cloud API / Twilio / Infobip	Front-end UI live (`/whatsapp` page). API integration **stubbed** — the `whatsapp` activity type is in the schema, the channel chip is in `messages.tsx` , but outbound send still needs a vendor key.
Voice (call center, dialer)	Twilio (planned)	Schema + UI live (`calls` , `power-dialer` , `voice-agents` pages). Real telephony integration **not yet wired**. The "live call" experience in `VoiceCallModal.tsx` uses the **browser's** Web Speech API for STT/TTS plus an LLM chat round-trip — no PSTN, no server-side Whisper.
SMS	Twilio (planned)	Schema + campaign channel enum (`sms`) live; vendor not yet wired.

---

9. Data layer

- **Database:** PostgreSQL (whatever Postgres URL is provided via env). Local dev uses Replit's built-in Postgres via the `database` skill.
- **ORM:** Drizzle (`drizzle-orm`), schema in `lib/db/src/schema/index.ts` .
- **Validation:** `drizzle-zod` (`createInsertSchema`) + **Zod v3** (catalog pin `^3.25.76` in `pnpm-workspace.yaml` , imported as plain `from "zod"` in `lib/db/src/schema/index.ts` and

the generated `lib/api-zod/src/generated/api/api.ts`). Schemas are re-exported by `lib/api-zod`.

- **Migrations:** Drizzle Kit, generated to `lib/db/drizzle/`. Apply via `drizzle-kit push` or `migrate` from a workspace script.
- **Auto-seed:** `artifacts/api-server/src/lib/autoSeed.ts` runs at boot. Populates a "default" org with sample contacts, companies, deals, calls, signals, agents. Idempotent — checks for existing rows before inserting.

---

10. Build & dev tooling

- **Package manager:** pnpm (workspace + catalog). No `--no-frozen-lockfile`.
- **Type system:** TypeScript 5.x, strict, project references for composite libs.
- **API codegen:** Orval, configured in `lib/api-spec/`. `pnpm --filter @workspace/api-spec run codegen` regenerates `lib/api-zod` + `lib/api-client-react`.
- **Linting / formatting:** Prettier + ESLint at the root.
- **Testing:** `vitest` available at the root for unit tests. End-to-end uses the workspace testing skill (Playwright-based subagent).

---

11. Runtime services & workflows

Each artifact is bound to a Replit workflow. Workflows are routed by path through the workspace shared proxy (mTLS, port 80):

PATH PREFIX	WORKFLOW	SERVICE	NOTES
`/`	`artifacts/nexflow: web`	Web SPA	Served by `vite` in dev, static in prod.
`/api`	`artifacts/api-server: API Server`	Express API	Listens on `$PORT`.
`/mobile` (and Expo dev domain)	`artifacts/mobile: expo`	Expo dev server	Uses `$REPLIT_EXPO_DEV_DOMAIN` directly, bypasses proxy.
`/marketing-demo-video/`	`artifacts/marketing-demo-video: web`	React video artifact	90 s scripted scenes.
`/__mockup`	`artifacts/mockup-sandbox: Component Preview Server`	Internal canvas	Designer-only.

Apps are exposed publicly on the domains in `$REPLIT_DOMAINS` over HTTPS via the proxy. There is **no** custom Vite proxy or per-service URL — everything goes through `localhost:80` in dev and the public domain in prod.

---

12. Environment variables (essential)

VARIABLE	USED BY	PURPOSE
`PORT`	api-server, mobile, web	Bind port (workflow-injected).
`DATABASE_URL`	`lib/db`	PostgreSQL connection.
`AI_INTEGRATIONS_OPENROUTER_BASE_URL` / `_API_KEY`	api-server	OpenRouter (preferred).
`AI_INTEGRATIONS_OPENAI_BASE_URL` / `_API_KEY`	api-server	OpenAI fallback + voice.
`RESEND_API_KEY` (or Replit *resend* connector)	api-server	Email.
`REPLIT_CONNECTORS_HOSTNAME`, `REPL_IDENTITY` / `WEB_REPL_RENEWAL`	api-server	Allows pulling secrets from Replit Connectors at runtime.
`REPLIT_DEV_DOMAIN`, `REPLIT_EXPO_DEV_DOMAIN`, `REPL_ID`	mobile	Expo tunnel.

All secrets must be set with the environment-secrets skill, never hardcoded.

---

13. Stack reconciliation (codebase vs. feasibility study)

The original feasibility study (`attached_assets/nexflow_feasibility_study_*.docx`) was written before code existed and used different stack choices in places. The **codebase wins** for engineering decisions; the feasibility study remains useful for pricing / market sizing arguments.

TOPIC	FEASIBILITY STUDY (BUSINESS PLAN)	CODEBASE (AUTHORITATIVE FOR BUILD)	RESOLUTION
Frontend framework	Next.js 15 (App Router, RSC)	**React 19 + Vite** SPA	Vite SPA shipped. Next.js was an early hypothesis when SSR was being considered for SEO. SEO is now scoped to the marketing site (`/marketing-demo-video/`), not the in-app dashboards, so RSC isn't needed. **Build wins: Vite.**
Mobile	"React Native (bare)"	**Expo SDK 54 / Expo Router**	Expo dramatically reduces native build surface and is what's actually shipping. **Build wins: Expo.**
Backend	"Node.js (NestJS or Express)"	**Express 5** with modular routers, no NestJS	Express 5 + modular routers + Drizzle is cleaner than NestJS for this scope. **Build wins: Express 5.**
ORM	"Prisma"	**Drizzle ORM**	Drizzle generates fewer artifacts and is friendlier to ESM bundling with esbuild. **Build wins: Drizzle.**
Validation	"class-validator (NestJS)"	**Zod v3** (`^3.25.76` in catalog) + `drizzle-zod`	Zod is shared between client and server via the generated `lib/api-zod` . **Build wins: Zod.**
Auth	"Auth0 / Cognito"	Currently demo-mode (no auth wall in the app)	Plan is to wire Replit Auth (or Clerk if customer requires SAML/SSO) before first paying customer. See `docs/04-deployment-plan.md` §5.
AI router	"Direct OpenAI calls"	**OpenRouter** (preferred) + OpenAI fallback	OpenRouter lets us A/B between Anthropic, Gemini, and OpenAI without code changes. **Build wins: OpenRouter.**
Voice	"Twilio Programmable Voice"	UI live; **telephony vendor not yet wired**	Twilio remains the planned vendor. To swap to Infobip / Avaya for GCC residency, the integration boundary is

TOPIC	FEASIBILITY STUDY (BUSINESS PLAN)	CODEBASE (AUTHORITATIVE FOR BUILD)	RESOLUTION
			`artifacts/api-server/src/routes/calls.ts`.
Hosting	"AWS me-south-1"	**Replit Deployments** (default), AWS me-south-1 (alternate path for residency)	See `docs/04-deployment-plan.md`.
Database	"AWS RDS Postgres"	Postgres (provider-agnostic via `DATABASE_URL`)	Replit Postgres in dev; Neon / Supabase / RDS in prod. **Build wins: provider-agnostic.**
Frontend i18n	"Arabic / English bilingual"	English UI today; Arabic content patterns appear in scripts (`scriptLanguageEnum: en | ar | both`) and in the assistant page	Arabic UI translation is an explicit Phase 1.5 deliverable, not in scope for this stack doc.

> **Rule:** when planning new work, the codebase choices in §1–§12 of this document are authoritative. If a customer-facing document still references Next.js / Prisma / NestJS, reconcile against this section.